



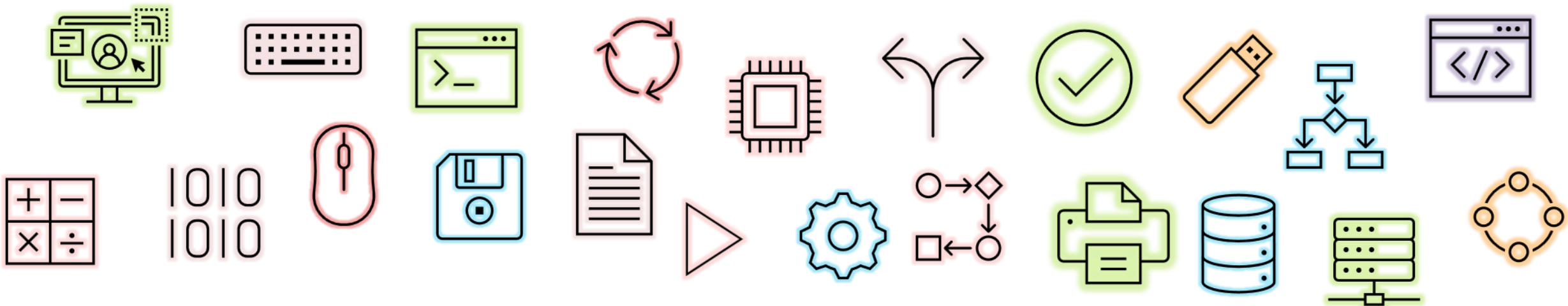
بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-202 Programming I

4- Selections



Objectives

- To declare boolean variables and write Boolean expressions using relational operators (§3.2).
- To implement selection control using one-way if statements (§3.3).
- To implement selection control using two-way if-else statements (§3.4).
- To implement selection control using nested if and multi-way if statements (§3.5).
- To avoid common errors and pitfalls in if statements (§3.6).
- To generate random numbers using the `Math.random()` method (§3.7).
- To program using selection statements for a variety of examples (SubtractionQuiz, BMI, ComputeTax) (§§3.7-3.9).
- To combine conditions using logical operators (`&&`, `||`, and `!`) (§3.10).
- To program using selection statements with combined conditions (LeapYear, Lottery) (§§3.11-3.12).
- To implement selection control using switch statements (§3.13).
- To write expressions using the conditional expression (§3.14).
- To examine the rules governing operator precedence and associativity (§3.15).
- To apply common techniques to debug errors (§3.16).

Motivations

- Consider this problem from topic 2:

LISTING 2.2 ComputeAreaWithConsoleInput.java

```
1 import java.util.Scanner; // Scanner is in the java.util package    import class
2
3 public class ComputeAreaWithConsoleInput {
4     public static void main(String[] args) {
5         // Create a Scanner object
6         Scanner input = new Scanner(System.in);                    create a Scanner
7
8         // Prompt the user to enter a radius
9         System.out.print("Enter a number for radius: ");
10        double radius = input.nextDouble();                        read a double
11
12        // Compute area
13        double area = radius * radius * 3.14159;
14
15        // Display results
16        System.out.println("The area for the circle of radius " +
17            radius + " is " + area);
18    }
19 }
```

Motivations

- What happens if the user inputs a negative value for radius?
 - The program would print an invalid result.
- If the radius is negative, you don't want the program to compute the area.
- How can you deal with this situation?

Motivations

- Java provides selection statements, which allow you to choose actions based on certain conditions
- For example, the following **selection** statement could be used in the previous program:

```
if (radius < 0) {  
  
}  
else {  
  
}
```

Motivations

- Java provides selection statements, which allow you to choose actions based on certain conditions
- For example, the following **selection** statement could be used in the previous program:

```
if (radius < 0) {  
    System.out.println("Incorrect input");  
}  
else {  
    area = radius * radius * 3.14159;  
    System.out.println("Area is " + area);  
}
```

Selection Statement

- Selection statements use conditions to test if something is **true** or **false**
 - These conditions are known as **Boolean expressions**
- **Boolean expression** is an expression that evaluates to a Boolean value
 - **true** or **false**
- We now introduce Boolean types and relational operators

The **boolean** Type and Operators

- Often in a program you need to compare two values, such as
 - Whether *i* is greater than *j*.
 - Or if an inputted value, such as *radius*, is less than zero.
- Java provides six relational operators AKA comparison operators that can be used to compare two values.

Relational Operators

Assume radius is **5** in the table.

TABLE 3.1 Relational Operators

<i>Java Operator</i>	<i>Mathematics Symbol</i>	<i>Name</i>	<i>Example (radius is 5)</i>	<i>Result</i>
<	<	less than	<code>radius < 0</code>	
<=	≤	less than or equal to	<code>radius <= 0</code>	
>	>	greater than	<code>radius > 0</code>	
>=	≥	greater than or equal to	<code>radius >= 0</code>	
==	=	equal to	<code>radius == 0</code>	
!=	≠	not equal to	<code>radius != 0</code>	

Caution

The equality testing operator is two equal signs (==), not a single equal sign (=). The latter symbol is for assignment.

Relational Operators

Assume radius is **5** in the table.

TABLE 3.1 Relational Operators

<i>Java Operator</i>	<i>Mathematics Symbol</i>	<i>Name</i>	<i>Example (radius is 5)</i>	<i>Result</i>
<	<	less than	<code>radius < 0</code>	<code>false</code>
<=	≤	less than or equal to	<code>radius <= 0</code>	
>	>	greater than	<code>radius > 0</code>	
>=	≥	greater than or equal to	<code>radius >= 0</code>	
==	=	equal to	<code>radius == 0</code>	
!=	≠	not equal to	<code>radius != 0</code>	

Caution

The equality testing operator is two equal signs (==), not a single equal sign (=). The latter symbol is for assignment.

Relational Operators

Assume radius is **5** in the table.

TABLE 3.1 Relational Operators

<i>Java Operator</i>	<i>Mathematics Symbol</i>	<i>Name</i>	<i>Example (radius is 5)</i>	<i>Result</i>
<	<	less than	radius < 0	false
<=	≤	less than or equal to	radius <= 0	false
>	>	greater than	radius > 0	true
>=	≥	greater than or equal to	radius >= 0	true
==	=	equal to	radius == 0	false
!=	≠	not equal to	radius != 0	true

Caution

The equality testing operator is two equal signs (==), not a single equal sign (=). The latter symbol is for assignment.

The **boolean** Type and Operators

- The result of comparing two values is a Boolean value: **true** or **false**.
- For example, the following prints **true**:
`System.out.println(2 > 1);`
- And the following prints **false**:
`System.out.println(3 > 7);`

The **boolean** Type and Operators

- A variable that holds a Boolean value is known as a **Boolean variable**
- The boolean data type is used to declare Boolean variables
 - A Boolean variable can hold one of two values
 - **true**
 - **false**
 - For example, the following statement assigns the value **true** to the variable **lightsOn**:

```
boolean lightsOn = true;
```

Check Point

- Assuming that **x** is **1**, show the result of the following Boolean expressions:

#	Expression	Result
a.	(x > 0)	
b.	(x < 0)	
c.	(x != 0)	
d.	(x >= 0)	
e.	(x != 1)	
f.	(x == 1)	

Check Point

- Assuming that **x** is **1**, show the result of the following Boolean expressions:

#	Expression	Result
a.	(x > 0)	true
b.	(x < 0)	
c.	(x != 0)	
d.	(x >= 0)	
e.	(x != 1)	
f.	(x == 1)	

Check Point

- Assuming that **x** is **1**, show the result of the following Boolean expressions:

#	Expression	Result
a.	(x > 0)	true
b.	(x < 0)	false
c.	(x != 0)	true
d.	(x >= 0)	true
e.	(x != 1)	false
f.	(x == 1)	true

Comparing and Testing Characters

- Two characters can be compared using relational operators
 - just like comparing two numbers
- This is done by comparing the **Unicode** values

Comparing and Testing Characters

- Two characters can be compared using relational operators
 - just like comparing two numbers
- This is done by comparing the **Unicode** values
- Examples:
 - `'a' < 'b'` is ...
 - Because ...
 - `'a' < 'A'` is ...
 - Because ...
 - `'1' < '8'` is ...
 - Because ...

Comparing and Testing Characters

- Two characters can be compared using relational operators
 - just like comparing two numbers
- This is done by comparing the **Unicode** values
- Examples:
 - `'a' < 'b'` is true
 - Because the Unicode for 'a' (97) is less than the Unicode for 'b' (98).
 - `'a' < 'A'` is ...
 - Because ...
 - `'1' < '8'` is ...
 - Because ...

Comparing and Testing Characters

- Two characters can be compared using relational operators
 - just like comparing two numbers
- This is done by comparing the **Unicode** values
- Examples:
 - `'a' < 'b'` is true
 - Because the Unicode for 'a' (97) is less than the Unicode for 'b' (98).
 - `'a' < 'A'` is false
 - Because the Unicode for 'a' (97) is greater than the Unicode for 'A' (65).
 - `'1' < '8'` is true
 - Because the Unicode for '1' (49) is less than the Unicode for '8' (56).

Check Point

- Evaluate the following :

Expression	Result
<code>System.out.println('a' < 'b');</code>	
<code>System.out.println('a' <= 'A');</code>	
<code>System.out.println('a' > 'b');</code>	
<code>System.out.println('a' >= 'A');</code>	
<code>System.out.println('a' == 'a');</code>	
<code>System.out.println('a' != 'b');</code>	

Check Point

- Evaluate the following :

Expression	Result
<code>System.out.println('a' < 'b');</code>	true
<code>System.out.println('a' <= 'A');</code>	
<code>System.out.println('a' > 'b');</code>	
<code>System.out.println('a' >= 'A');</code>	
<code>System.out.println('a' == 'a');</code>	
<code>System.out.println('a' != 'b');</code>	

Check Point

- Evaluate the following :

Expression	Result
<code>System.out.println('a' < 'b');</code>	true
<code>System.out.println('a' <= 'A');</code>	false
<code>System.out.println('a' > 'b');</code>	false
<code>System.out.println('a' >= 'A');</code>	true
<code>System.out.println('a' == 'a');</code>	true
<code>System.out.println('a' != 'b');</code>	true

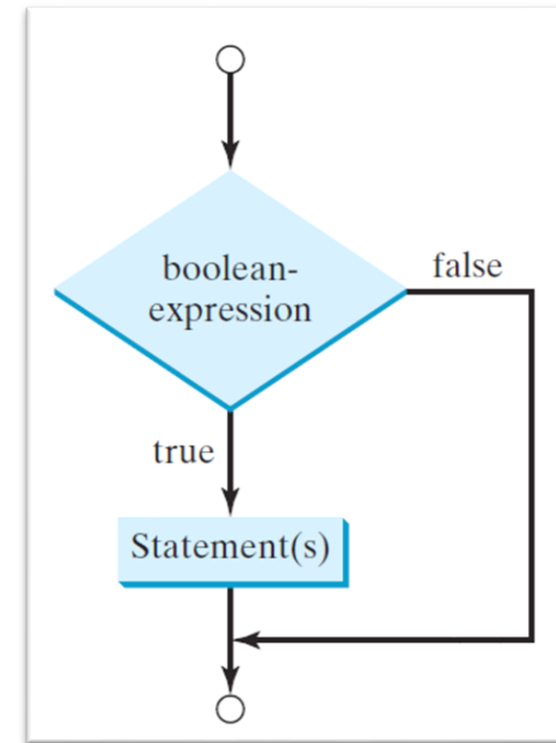
if Statements

- The previous program would display a message such as “6+2 = 7” is “false”
- What if you wanted to print something else, such as “6+2 = 7” is “incorrect”
- How would you do this?
 - Answer: selection statements!
- Java has several types of selection statements:
 - One-way **if** statements
 - Two-way **if-else** statements
 - Nested **if** statements
 - Multi-way **if-else** statements
 - **switch** statements, and conditional expressions

One-way **if** Statements

- A one-way if statement executes an action iff the condition is true
- Syntax:

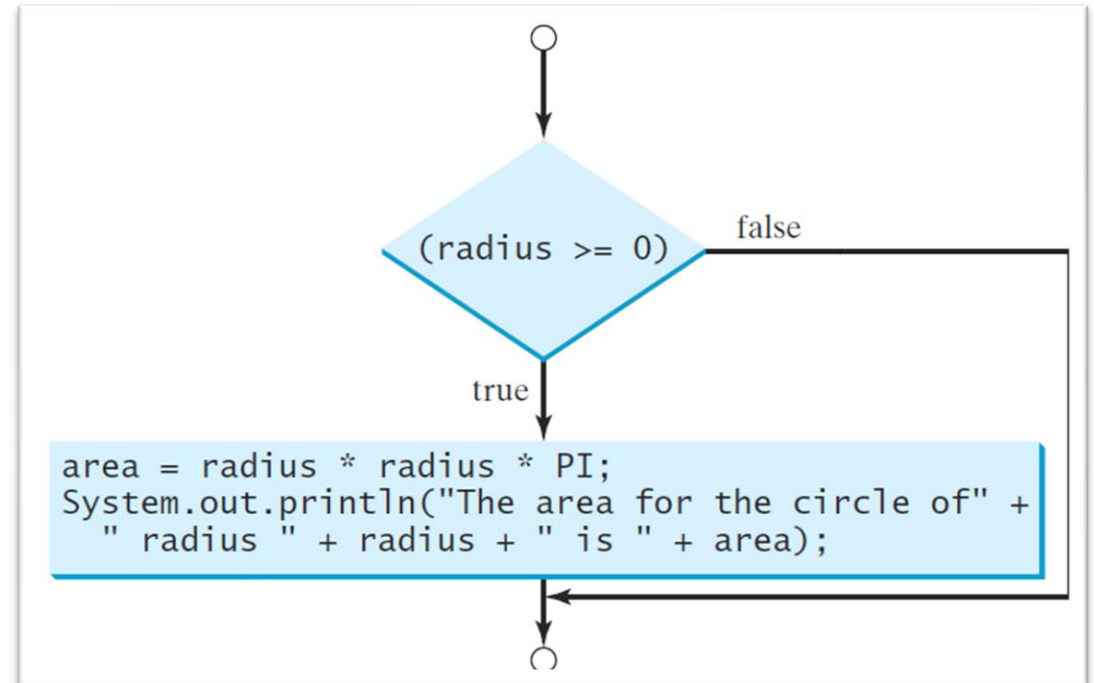
```
if (boolean-expression) {  
    statement(s);  
}
```
- The flowchart to the right demonstrates the syntax of an **if** statement.



One-way **if** Statements

- So if the boolean-expression evaluates to **true**, the statements in the block are executed
- Example

```
if (radius >= 0) {  
    area = radius * radius * PI;  
    System.out.println("The area " +  
        "for the circle " +  
        "of radius " + radius +  
        " is " + area);  
}
```



Note

- The boolean-expression should be enclosed in parenthesis

```
if i > 0 {  
    System.out.println("i is positive");  
}
```

- The block braces can be removed if they enclose a single

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

Note

- The boolean-expression should be enclosed in parenthesis:

```
if i > 0 {  
    System.out.println("i is positive");  
}
```

Wrong

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

Correct

- The block braces can be removed if they enclose a single

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

Note

- The boolean-expression should be enclosed in parenthesis:

```
if i > 0 {  
    System.out.println("i is positive");  
}
```

Wrong

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

Correct

- The block braces can be removed if they enclose a single

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

```
if (i > 0)  
    System.out.println("i is positive");
```

Equivalent

Program 1: Simple `if` Demo

- Write a program that prompts the user to enter an integer. If the number is a multiple of 5, the program should print `HiFive`. If the number is divisible by 2, the program should print `HiEven`.
- Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 1: Simple `if` Demo

- Step 1 : Problem-solving Phase
 - Algorithm:
 1. Prompt the user to enter an integer
 2. Scan the value from user and save to variable `number`
 3. If number is a multiple of 5, print `HiFive`.
`if (number % 5 == 0)`
 4. If number is divisible by 2, print `HiEven`.
`if (number % 2 == 0)`

Program 1: Simple `if` Demo

- Step 1 : Problem-solving Phase
 - Algorithm:
 1. Prompt the user to enter an integer
 2. Scan the value from user and save to variable `number`
 3. If number is a multiple of 5, print `HiFive`.
`if (number % 5 == 0)`
 4. If number is divisible by 2, print `HiEven`.
`if (number % 2 == 0)`

Program 1: Simple `if` Demo

- Run the Program



Enter an integer:

Program 1: Simple `if` Demo

- Run the Program



```
Enter an integer: 4
```

Program 1: Simple `if` Demo

- Run the Program



Enter an integer: 4
HiEven

Program 1: Simple `if` Demo

- Run the Program



```
Enter an integer: 4 ↵ Enter  
HiEven
```



```
Enter an integer:
```

Program 1: Simple `if` Demo

- Run the Program



```
Enter an integer: 4 ↵ Enter  
HiEven
```



```
Enter an integer: 30 |
```

Program 1: Simple `if` Demo

- Run the Program



```
Enter an integer: 4 ↵ Enter  
HiEven
```



```
Enter an integer: 30 ↵ Enter  
HiFive  
HiEven
```

Check Point

- Write an `if` statement that increases pay by 3% if score is greater than 90.

```
if (score > 90) {  
  
}
```

or

```
if (score > 90) {  
  
}
```

Check Point

- Write an `if` statement that increases pay by 3% if score is greater than 90.

```
if (score > 90) {  
    pay = pay + pay * 0.03;  
}
```

or

```
if (score > 90) {  
  
}
```

Check Point

- Write an `if` statement that increases `pay` by 3% if `score` is greater than 90.

```
if (score > 90) {  
    pay = pay + pay * 0.03;  
}
```

or

```
if (score > 90) {  
    pay *= 1.03;    // same as  
}                  // pay = pay * 1.03;
```

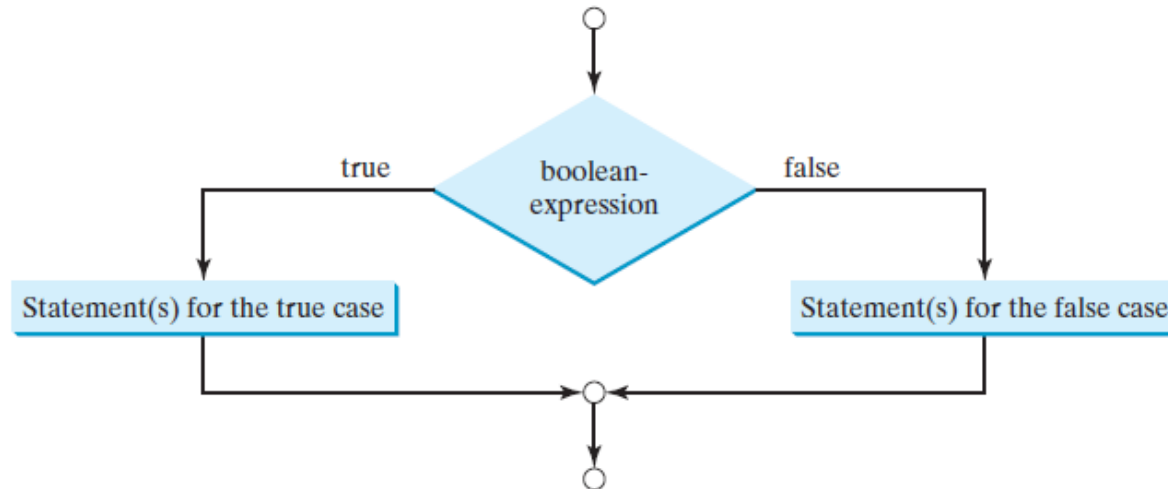
Two-way if-else Statements

- A one-way if statement performs an action if the given condition is **true**.
 - If the condition is **false**, nothing is done.
- But what if you want to take alternative actions when the condition is **false**?
 - Answer: use a two-way **if-else** statement.
- The actions specified by a two-way **if-else** statement differ based on whether the condition is **true** or **false**

Two-way if-else Statements

- Syntax:

```
if (boolean-expression) {  
    statement(s)-for-the-true-case;  
}  
else {  
    statement(s)-for-the-false-case;  
}
```



Two-way if-else Statements

- If the boolean-expression evaluates to true, the statement(s) for the true case are executed
- Else, the statement(s) for the false case are executed
- Example:

```
if (radius >= 0) {  
    area = radius * radius * PI;  
    System.out.println("The area for the circle of radius " +  
        radius + " is " + area);  
}  
else {  
    System.out.println("Negative input");  
}
```

Two-way if-else Statements

- Another example:
 - Check whether a number is even or odd

```
if (number % 2 == 0)
    System.out.println(number + " is even.");
else
    System.out.println(number + " is odd.");
```

Comparing and Testing Characters

- Often you must test whether a character is a number, a letter, or even if it is uppercase or lowercase.
- The following code tests whether a character `ch` is an uppercase letter, a lowercase letter, or a digit:

```
if (ch >= 'A' && ch <= 'Z')  
    System.out.println(ch + " is an uppercase letter");  
else if (ch >= 'a' && ch <= 'z')  
    System.out.println(ch + " is a lowercase letter");  
else if (ch >= '0' && ch <= '9')  
    System.out.println(ch + " is a numeric character");
```

Check Point

- Write an `if` statement that increases pay by 3% if score is greater than 90, otherwise increases pay by only 1%.
- Solution:

Check Point

- Write an if statement that increases pay by 3% if score is greater than 90, otherwise increases pay by only 1%.
- Solution:

```
if (score > 90) {  
    pay *= 1.03;  
else  
    pay *= 1.01;  
}
```

Nested `if` and Multi-Way `if-else` Statements

- The statement inside an `if` or `if-else` statement can be any legal Java statement
 - Including another `if` or `if-else` statement!
- The inner `if` statement is said to be `nested` inside the outer `if` statement
- Example

Nested **if** and Multi-Way **if-else** Statements

- The statement inside an **if** or **if-else** statement can be any legal Java statement
 - Including another **if** or **if-else** statement!
- The inner **if** statement is said to be **nested** inside the outer **if** statement

This statement is nested inside the **if (i > k)** statement

- Example

```
if (i > k) {  
    if (j > k)  
        System.out.println("i and j are greater than k");  
}  
else  
    System.out.println("i is less than or equal to k");  
}
```

Nested **if** and Multi-Way **if-else** Statements

- More details:
 - The inner **if** statement can contain another **if** statement
 - In fact, there is no limit to the depth of the nesting
- So what is the purpose?
 - The nested **if** statement can be used to implement multiple alternatives
 - Consider the following example, which prints a letter grade according to the final number grade

Nested **if** and Multi-Way **if-else** Statements

```
if (score >= 90.0)
    System.out.print("A");
else
    if (score >= 80.0)
        System.out.print("B");
    else
        if (score >= 70.0)
            System.out.print("C");
        else
            if (score >= 60.0)
                System.out.print("D");
            else
                System.out.print("F");
```

(a)

Nested **if** and Multi-Way **if-else** Statements

```
if (score >= 90.0)
    System.out.print("A");
else
    if (score >= 80.0)
        System.out.print("B");
    else
        if (score >= 70.0)
            System.out.print("C");
        else
            if (score >= 60.0)
                System.out.print("D");
            else
                System.out.print("F");
```

(a)

Equivalent

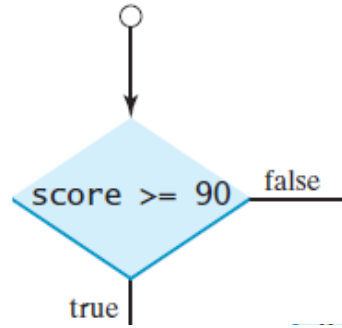
This is better

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D");
else
    System.out.print("F");
```

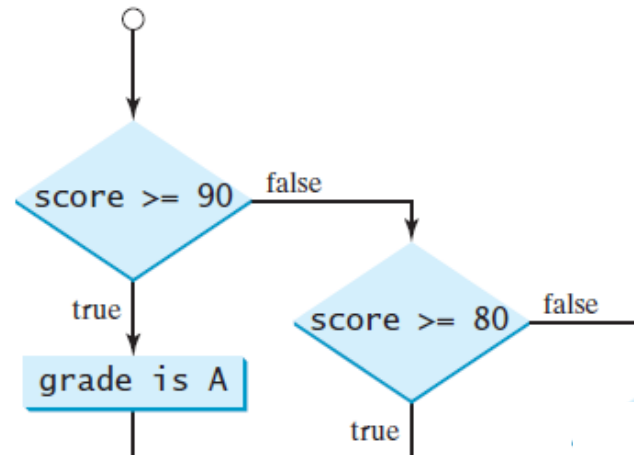
(b)

- While (a) works, the preferred format for multiple alternatives is shown in (b) using a multi-way if-else statement
- This multi-way if-else style avoids deep indentation and makes the program easier to read

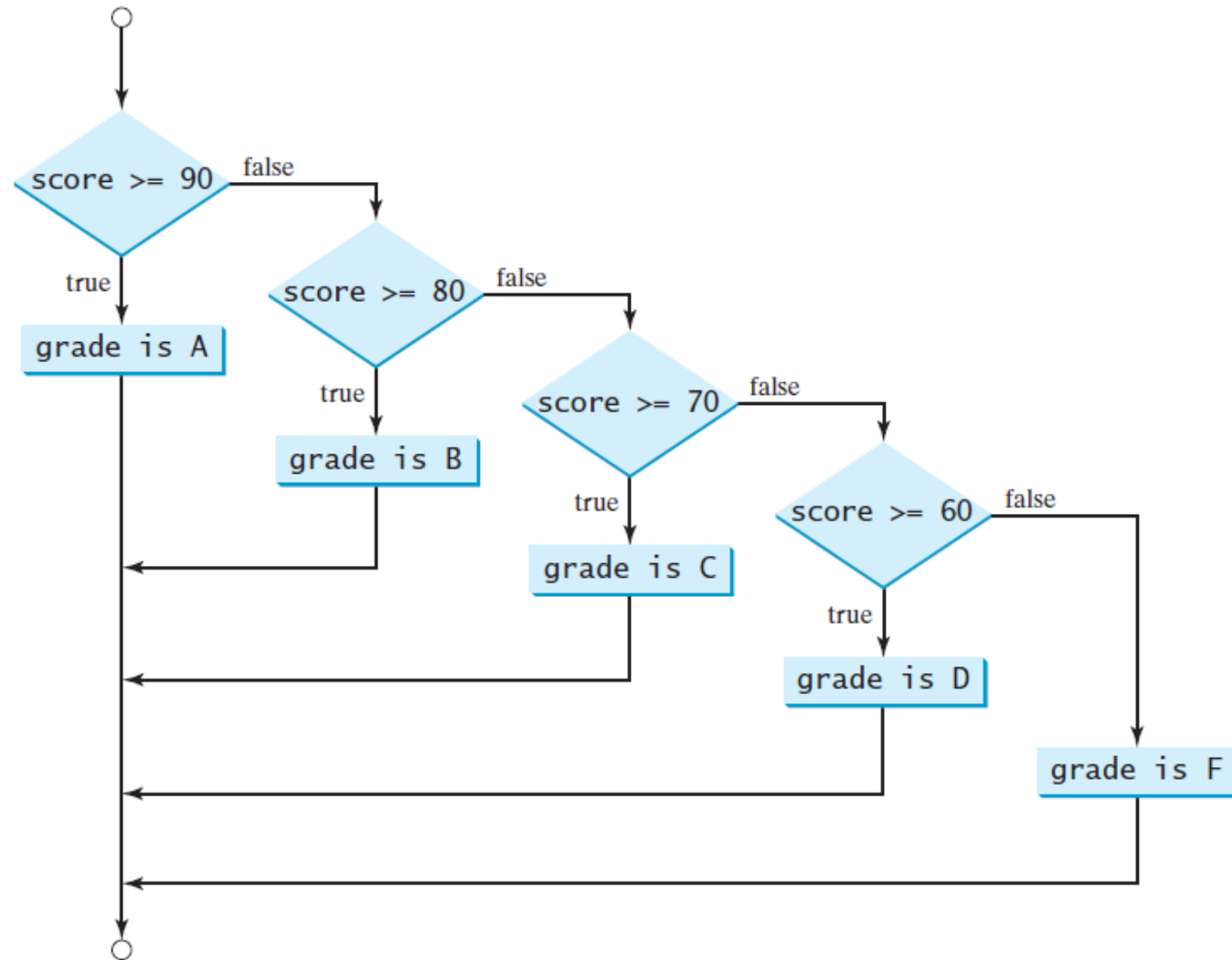
Nested **if** and Multi-Way **if-else** Statements



Nested **if** and Multi-Way **if-else** Statements



Nested **if** and Multi-Way **if-else** Statements



Trace **if-else** Statement

Suppose score is 70.0

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D"); else
System.out.print("F");
```

Trace **if-else** Statement

Suppose score is 70.0

Condition is false

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D"); else
System.out.print("F");
```

Trace **if-else** Statement

Suppose score is 70.0

Condition is false

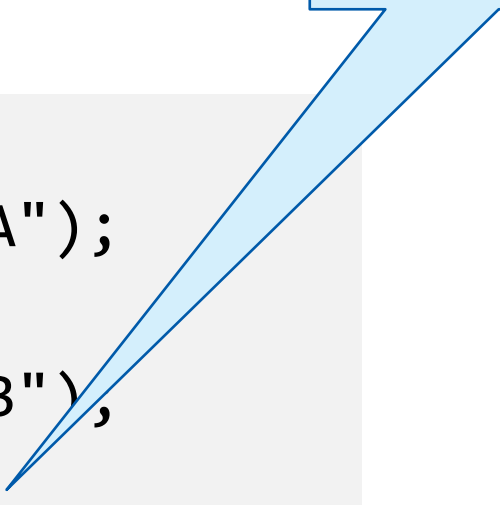
```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D"); else
System.out.print("F");
```

Trace **if-else** Statement

Suppose score is 70.0

Condition is true

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D"); else
System.out.print("F");
```

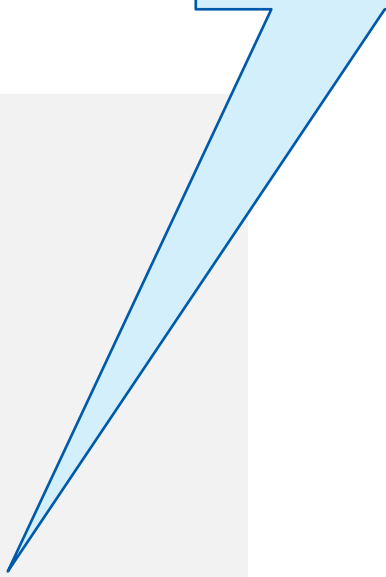


Trace **if-else** Statement

Suppose score is 70.0

grads is C

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D"); else
System.out.print("F");
```

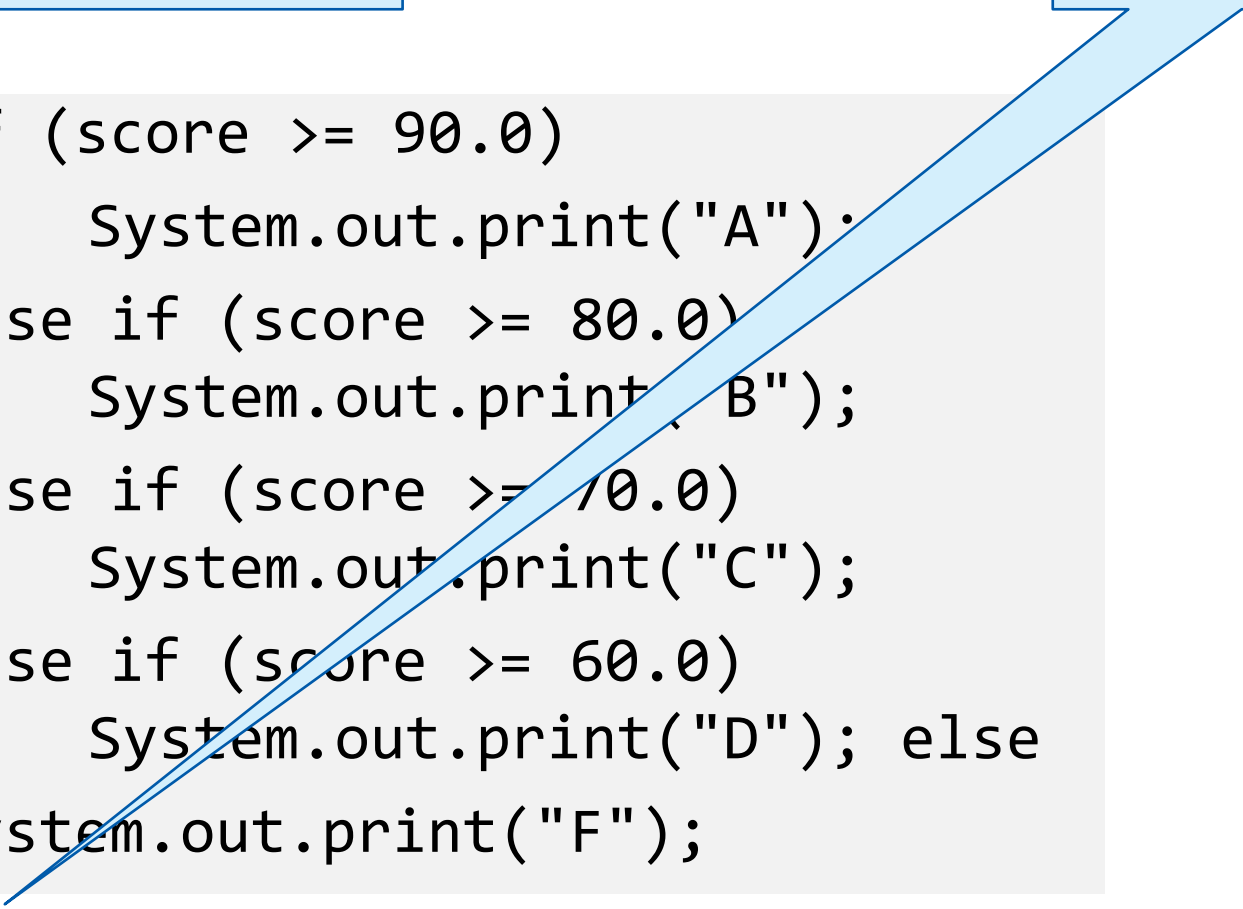


Trace **if-else** Statement

Suppose score is 70.0

Exit the if statement

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D"); else
    System.out.print("F");
...
```



Trace **if-else** Statement

Suppose score is 70.0

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D"); else
    System.out.print("F");
```

...

Note:

A condition is only tested when all the conditions that come before it are false.

Check Point

- Given the following code, show the output when:
 - $x = 2$ and $y = 3$
 - $x = 3$ and $y = 2$
 - $x = 3$ and $y = 3$

```
if (x > 2) {  
    if (y > 2) {  
        z = x + y;  
        System.out.println("z is " + z);  
    }  
}  
else  
    System.out.println("x is " + x);
```

Common Errors and Pitfalls

- Common Error 1: Forgetting Necessary Braces
 - Remember: braces can be removed IF the block only contains a single statement (only one statement)
 - However, if there are multiple statements, brackets are required.
 - A common error is to forget these required braces when grouping multiple lines together

Common Errors and Pitfalls

- Common Error 1: Forgetting Necessary Braces

```
if (radius >= 0)
    area = radius * radius * PI;
System.out.println("The area "
    + " is " + area);
```

Wrong

```
if (radius >= 0) {
    area = radius * radius * PI;
System.out.println("The area "
    + " is " + area);
}
```

Correct

- a. Wrong
 - Braces are required
- b. Correct
 - Required braces are now included

Common Errors and Pitfalls

- Common Error 2: Wrong Semicolon at the **if** Line
 - There should **not** be a semicolon at the end of the **if** line.
 - This is a common mistake new programmers make.
- And this mistake is hard to find, because it is a logic error.

Logic error

```
if (radius >= 0);  
{  
    area = radius * radius * PI;  
    System.out.println("The area "  
        + " is " + area);  
}
```

Equivalent

Empty block

```
if (radius >= 0) {}  
{  
    area = radius * radius * PI;  
    System.out.println("The area "  
        + " is " + area);  
}
```

Common Errors and Pitfalls

- Common Error 3: Dangling **else** Ambiguity
 - The code in (a) below has two **if** clauses and one else clause.
 - Which **if** clause is matched by the **else** clause?

```
int i = 1, j = 2, k = 3;  
  
if (i > j)  
    if (i > k)  
        System.out.println("A");  
else  
    System.out.println("B");
```

(a)

Common Errors and Pitfalls

- Common Error 3: Dangling **else** Ambiguity
 - The code in (a) below has two **if** clauses and one else clause.
 - Which **if** clause is matched by the **else** clause?
 - The indentation suggests that the else clause matches the first **if** clause.
 - However, this is NOT correct.

```
int i = 1, j = 2, k = 3;
if (i > j)
    if (i > k)
        System.out.println("A");
    else
        System.out.println("B");
```

(a)

Common Errors and Pitfalls

- Common Error 3: Dangling **else** Ambiguity
 - The code in (a) below has two **if** clauses and one else clause.
 - Which **if** clause is matched by the **else** clause?
 - The **else** clause actually matches the second **if** clause.

```
int i = 1, j = 2, k = 3;  
  
if (i > j)  
    if (i > k)  
        System.out.println("A");  
    else  
        System.out.println("B");
```

(a)

Common Errors and Pitfalls

- Common Error 3: Dangling **else** Ambiguity
 - The code in (a) below has two **if** clauses and one **else** clause.
 - Which **if** clause is matched by the **else** clause?
 - The **else** clause actually matches the second **if** clause.
 - The **else** clause always matches the most recent unmatched **if** clause in the

```
int i = 1, j = 2, k = 3;
if (i > j)
    if (i > k)
        System.out.println("A");
    else
        System.out.println("B");
```

(a)

Equivalent

This is better
with correct
indentation →

```
int i = 1, j = 2, k = 3;
if (i > j)
    if (i > k)
        System.out.println("A");
    else
        System.out.println("B");
```

(b)

Common Errors and Pitfalls

- Common Error 3: Dangling **else** Ambiguity
 - If you actually wanted the **else** clause to match with the first **if** clause, then you must add a pair of **braces**
 - This code displays **B**.

```
int i = 1, j = 2, k = 3;
if (i > j) {
    if (i > k)
        System.out.println("A");
}
else
    System.out.println("B");
```

Common Errors and Pitfalls

- Common Pitfall 1: Simplifying Boolean Variable Assignment
 - Often, new programmers write code like (a).

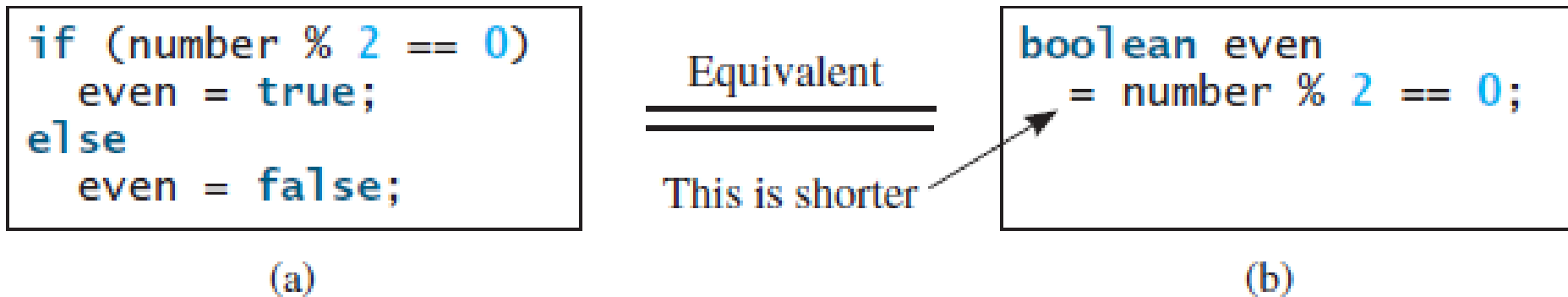
```
if (number % 2 == 0)
    even = true;
else
    even = false;
```

(a)

- This is not an error

Common Errors and Pitfalls

- Common Pitfall 1: Simplifying Boolean Variable Assignment
 - Often, new programmers write code like (a).



- This is not an error
- But it is better written (and shorter) as shown in (b).

Common Errors and Pitfalls

- Common Pitfall 2: Avoid Duplicating Code in Different Cases
 - Often, new programmers write duplicate code that should be combined in one place.

```
if (inState) {  
    tuition = 5000;  
    System.out.println("The tuition is " + tuition);  
}  
else {  
    tuition = 15000;  
    System.out.println("The tuition is " + tuition);  
}
```

- This is not an error, but it is duplicating efforts.

Common Errors and Pitfalls

- Common Pitfall 2: Avoid Duplicating Code in Different Cases
 - Often, new programmers write duplicate code that should be combined in one place.

```
if (inState) {  
    tuition = 5000;  
    System.out.println("The tuition is " + tuition);  
}  
else {  
    tuition = 15000;  
    System.out.println("The tuition is " + tuition);  
}
```

- This is not an error, but it is duplicating efforts.

Common Errors and Pitfalls

- Common Pitfall 2: Avoid Duplicating Code in Different Cases
 - It is better written as follows:

```
if (inState) {  
    tuition = 5000;  
}  
else {  
    tuition = 15000;  
}  
System.out.println("The tuition is " + tuition);
```

Common Errors and Pitfalls

- Common Pitfall 2: Avoid Duplicating Code in Different Cases
 - It is better written as follows:

```
if (inState)
    tuition = 5000;
else
    tuition = 15000;
System.out.println("The tuition is " + tuition);
```

Check Point

- Rewrite the following statement using a Boolean expression:

```
if (count % 10 == 0)
    newline = true;
else
    newline = false;
```

- Solution

Check Point

- Rewrite the following statement using a Boolean expression:

```
if (count % 10 == 0)
    newline = true;
else
    newline = false;
```

- Solution

```
newline = count % 10 == 0;
```

Check Point

- What is the output of the following code if number is 14, 15, or 30?

```
if (number % 2 == 0)
    System.out.println
        (number + " is even");
if (number % 5 == 0)
    System.out.println
        (number + " is multiple of 5");
```

(a)

```
if (number % 2 == 0)
    System.out.println
        (number + " is even");
else if (number % 5 == 0)
    System.out.println
        (number + " is multiple of 5");
```

(b)

Program 2: Math Learning Tool⁺

- Write a program that helps a first-grader practice subtraction.
 - The program should randomly generate two single-digit integers and should then ask the user for the answer. The program will then display a message stating if the answer is correct. If wrong, the program should display the correct answer.
- Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 2: Math Learning Tool⁺

- Step 1 : Problem-solving Phase
 - How can we generate random numbers?
 - We used `currentTimeMillis()` previously
 - A better approach is to use the `random()` method, which Java provides for you in the `Math` class.
 - If you call this method, it will return a random `double` value, `d`, ($0.0 < d < 1.0$)
 - Examples: 0.832 or 0.258, or 0.927, or 0.004
 - So from these numbers, how do we get a single digit `int`
 - A number between 0 and 9
 - Easy: we multiply it by 10 and then cast it as an `int`
`int number1 = (int)(Math.random()*10);`

Program 2: Math Learning Tool⁺

- Step 1 : Problem-solving Phase
- So here is our **algorithm**:
 1. **Generate** two **random** numbers
 - `int number1 = (int)(Math.random()*10);`
 - `int number2 = (int)(Math.random()*10);`
 2. Make sure that `number1` is larger than `number2`. If not, **swap** them!
 3. **Ask** the user to answer a question
 - Example: “What is 6 - 2?”
 4. **Scan** the user’s answer
 5. **Print** whether the answer is correct.

Program 2: Math Learning Tool⁺

- Step 2: Implementation

LISTING 3.3 SubtractionQuiz.java

```
1 import java.util.Scanner;
2
3 public class SubtractionQuiz {
4     public static void main(String[] args) {
5         // 1. Generate two random single-digit integers
6         int number1 = (int)(Math.random() * 10);
7         int number2 = (int)(Math.random() * 10);
8
9         // 2. If number1 < number2, swap number1 with number2
10        if (number1 < number2) {
11            int temp = number1;
12            number1 = number2;
13            number2 = temp;
14        }
15
16        // 3. Prompt the student to answer "What is number1 - number2?"
17        System.out.print
18            ("What is " + number1 + " - " + number2 + "? ");
19        Scanner input = new Scanner(System.in);
20        int answer = input.nextInt();
21
22        // 4. Grade the answer and display the result
23        if (number1 - number2 == answer)
24            System.out.println("You are correct!");
25        else {
26            System.out.println("Your answer is wrong.");
27            System.out.println(number1 + " - " + number2 +
28                " should be " + (number1 - number2));
29        }
30    }
31 }
```

random number

Program 2: Math Learning Tool⁺

- Run the Program:



What is 6 - 6? |

Program 2: Math Learning Tool⁺

- Run the Program:



```
What is 6 - 6? 0
```

Program 2: Math Learning Tool⁺

- Run the Program:



What is 6 - 6? 0 Enter
You are correct!

Program 2: Math Learning Tool⁺

- Run the Program:



What is 6 - 6? 0
You are correct!



What is 9 - 2?

Program 2: Math Learning Tool⁺

- Run the Program:



What is 6 - 6? 0
You are correct!



What is 9 - 2? 5

Program 2: Math Learning Tool⁺

- Run the Program:



What is 6 - 6? 0 ↵ Enter
You are correct!



What is 9 - 2? 5 ↵ Enter
Your answer is wrong
9 - 2 is 7

Check Point

- How do you generate a random integer i , such that $0 \leq i < 10$?
- How do you generate a random integer i , such that $0 \leq i < 20$?
- How do you generate a random integer i , such that $10 \leq i < 20$?
- How do you generate a random integer i , such that $10 \leq i < 50$?
- Write an expression that returns 0 or 1 randomly.

Check Point

- How do you generate a random integer i , such that $0 \leq i < 10$?
`int number = (int)(Math.random()*10);`
- How do you generate a random integer i , such that $0 \leq i < 20$?
- How do you generate a random integer i , such that $10 \leq i < 20$?
- How do you generate a random integer i , such that $10 \leq i < 50$?
- Write an expression that returns 0 or 1 randomly.

Check Point

- How do you generate a random integer i , such that $0 \leq i < 10$?
`int number = (int)(Math.random()*10);`
- How do you generate a random integer i , such that $0 \leq i < 20$?
`int number = (int)(Math.random()*20);`
- How do you generate a random integer i , such that $10 \leq i < 20$?
- How do you generate a random integer i , such that $10 \leq i < 50$?
- Write an expression that returns 0 or 1 randomly.

Check Point

- How do you generate a random integer i , such that $0 \leq i < 10$?
`int number = (int)(Math.random()*10);`
- How do you generate a random integer i , such that $0 \leq i < 20$?
`int number = (int)(Math.random()*20);`
- How do you generate a random integer i , such that $10 \leq i < 20$?
`int number = 10 + (int)(Math.random()*10);`
- How do you generate a random integer i , such that $10 \leq i < 50$?
- Write an expression that returns 0 or 1 randomly.

Check Point

- How do you generate a random integer i , such that $0 \leq i < 10$?
`int number = (int)(Math.random()*10);`
- How do you generate a random integer i , such that $0 \leq i < 20$?
`int number = (int)(Math.random()*20);`
- How do you generate a random integer i , such that $10 \leq i < 20$?
`int number = 10 + (int)(Math.random()*10);`
- How do you generate a random integer i , such that $10 \leq i < 50$?
`int number = 10 + (int)(Math.random()*40);`
- Write an expression that returns 0 or 1 randomly.

Check Point

- How do you generate a random integer i , such that $0 \leq i < 10$?
`int number = (int)(Math.random()*10);`
- How do you generate a random integer i , such that $0 \leq i < 20$?
`int number = (int)(Math.random()*20);`
- How do you generate a random integer i , such that $10 \leq i < 20$?
`int number = 10 + (int)(Math.random()*10);`
- How do you generate a random integer i , such that $10 \leq i < 50$?
`int number = 10 + (int)(Math.random()*40);`
- Write an expression that returns 0 or 1 randomly.
`int number = (int)(Math.random()*2);`

Program 3: Computing BMI

- Write a program that computes the **Body Mass Index (BMI)** for the user. Your program should prompt the user to enter a weight in pounds and height in inches. Your program should then compute the BMI and display for the user.
- Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 3: Computing BMI

- Step 1: Problem-solving Phase

- Notes:

- BMI is a measure of health based on the height and weight
 - BMI is calculated by taking the weight (in kilograms) and then dividing it by the square of the height (in meters)

$$BMI = \frac{weightInKilograms}{heightinMeters \times heightinMeters}$$

- The interpretation

BMI	Interpretation
BMI < 18.5	Underweight
18.5 ≤ BMI < 25.0	Normal
25.0 ≤ BMI < 30.0	Overweight
30.0 ≤ BMI	Obese

er is as follows:

Program 3: Computing BMI

- Step 1: Problem-solving Phase
 - Notes:
 - So the user input is in pounds and inches
 - The BMI equation is in kilograms and meters
 - Therefore, you will need to convert from:
 - pounds to kilograms
 - » One pound is 0.45359237 kilograms
 - inches to meters
 - » one inch is 0.0254 meters

Program 3: Computing BMI

- Step 1: Problem-solving Phase
- Algorithm:
 1. Prompt user for input
 2. Scan height and weight from user
 3. Compute BMI using BMI equation
 4. Display results to user

Program 3: Computing BMI

- Note the use of constants in the program
 - Makes the conversion formulas easier to read and understand

LISTING 3.4 ComputeAndInterpretBMI.java

```
1  import java.util.Scanner;
2
3  public class ComputeAndInterpretBMI {
4      public static void main(String[] args) {
5          Scanner input = new Scanner(System.in);
6
7          // Prompt the user to enter weight in pounds
8          System.out.print("Enter weight in pounds: ");
9          double weight = input.nextDouble();           input weight
10
11         // Prompt the user to enter height in inches
12         System.out.print("Enter height in inches: ");
13         double height = input.nextDouble();           input height
14
15         final double KILOGRAMS_PER_POUND = 0.45359237; // Constant
16         final double METERS_PER_INCH = 0.0254; // Constant
17
18         // Compute BMI
19         double weightInKilograms = weight * KILOGRAMS_PER_POUND;
20         double heightInMeters = height * METERS_PER_INCH;
21         double bmi = weightInKilograms /              compute bmi
22             (heightInMeters * heightInMeters);
23
24         // Display result
25         System.out.println("BMI is " + bmi);           display output
26         if (bmi < 18.5)
27             System.out.println("Underweight");
28         else if (bmi < 25)
29             System.out.println("Normal");
30         else if (bmi < 30)
31             System.out.println("Overweight");
32         else
33             System.out.println("Obese");
34     }
```

Program 3: Computing BMI

- Run the Program:



Enter weight in pounds:

Program 3: Computing BMI

- Run the Program:



```
Enter weight in pounds: 146
```

Program 3: Computing BMI

- Run the Program:



```
Enter weight in pounds: 146 ↵ Enter  
Enter height in inches:
```

Program 3: Computing BMI

- Run the Program:



```
Enter weight in pounds: 146 ↵ Enter  
Enter height in inches: 70
```

Program 3: Computing BMI

- Run the Program:



```
Enter weight in pounds: 146 ↵ Enter  
Enter height in inches: 70 ↵ Enter  
BMI is 20.948603801493316  
Normal
```

Logical Operators

- We have used conditional statements to help us determine if the execution should take one path (true path) or another path (false path)
 - But until now, these conditional statements have been very basic
- Usually, whether a statement is executed is determined by a **combination** of several conditions.
 - You can use logical operators to combine these conditions to form a compound Boolean expression.

Logical Operators

- Logical operators, also known as Boolean operators, operate on Boolean values to create a new Boolean value
- The following page shows the four logical operators we will use
- The four pages after that show a truth table for each logical operator and some examples

Logical Operators

TABLE 3.3 Boolean Operators

<i>Operator</i>	<i>Name</i>	<i>Description</i>
!	not	logical negation
&&	and	logical conjunction
	or	logical disjunction
^	exclusive or	logical exclusion

Truth Table for Operator !

TABLE 3.4 Truth Table for Operator !

p	!p	<i>Example (assume age = 24, weight = 140)</i>
true	false	!(age > 18) is false, because (age > 18) is true.
false	true	!(weight == 150) is true, because (weight == 150) is false.

Truth Table for Operator &&

TABLE 3.5 Truth Table for Operator &&

p ₁	p ₂	p ₁ && p ₂	<i>Example (assume age = 24, weight = 140)</i>
false	false	false	
false	true	false	(age > 28) && (weight <= 140) is true, because (age > 28) is false.
true	false	false	
true	true	true	(age > 18) && (weight >= 140) is true, because (age > 18) and (weight >= 140) are both true.

Truth Table for Operator ||

TABLE 3.6 Truth Table for Operator ||

p ₁	p ₂	p ₁ p ₂	Example (assume age = 24, weight = 140)
false	false	false	(age > 34) (weight >= 150) is false, because (age > 34) and (weight >= 150) are both false.
false	true	true	
true	false	true	(age > 18) (weight < 140) is true, because (age > 18) is true.
true	true	true	

Truth Table for Operator $\wedge$

TABLE 3.7 Truth Table for Operator $\wedge$

p_1	p_2	$p_1 \wedge p_2$	Example (assume $\text{age} = 24$, $\text{weight} = 140$)
false	false	false	$(\text{age} > 34) \wedge (\text{weight} > 140)$ is false, because $(\text{age} > 34)$ and $(\text{weight} > 140)$ are both false.
false	true	true	$(\text{age} > 34) \wedge (\text{weight} \geq 140)$ is true, because $(\text{age} > 34)$ is false but $(\text{weight} \geq 140)$ is true.
true	false	true	
true	true	false	

Program 4: TestBooleanOperators

- Write a program that tests the usage of boolean operators. Specifically, your program should prompt the user to enter one integer. Your program should then determine if the value is divisible by 2 and 3, by 2 or 3, or by 2 or 3 but not both.
- Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 4: TestBooleanOperators

- Step 1 : Problem-solving Phase
 - Notes:
 - What does it mean “the value is divisible by”?
 - If a number is divisible by some other number, this means that it divides perfectly into that number with no remainder
 - Example:
 - 15 is divisible by 3 and also by 5.
 - However, 15 is not divisible by 4.
 - Example:
 - 4 is divisible by 2.
 - 4 is not divisible by 3.

Program 4: TestBooleanOperators

- Step 1 : Problem-solving Phase
 - Notes:
 - So how do we check for divisibility?
 - We use mod (%)
 - Example: check if some number, x , is divisible by 3
`if (x % 3 == 0)`
 - This says: if we divide by 3 and the remainder is zero...
 - And that is exactly what we want!
 - However, we must check the divisibility of two numbers
 - both 2 and 3
 - This means we must use logical operators

Program 4: TestBooleanOperators

- Step 1 : Problem-solving Phase
- Algorithm:
 1. Prompt user for input
 2. Scan the integer value from user
 3. Determine the divisibility conditions
 4. Display results to user

Program 4: TestBooleanOperators

- Step 2:
Implementation

LISTING 3.6 TestBooleanOperators.java

```
1  import java.util.Scanner;
2
3  public class TestBooleanOperators {
4      public static void main(String[] args) {
5          // Create a Scanner
6          Scanner input = new Scanner(System.in);
7
8          // Receive an input
9          System.out.print("Enter an integer: ");
10         int number = input.nextInt();
11
12         if (number % 2 == 0 && number % 3 == 0)
13             System.out.println(number + " is divisible by 2 and 3.");
14
15         if (number % 2 == 0 || number % 3 == 0)
16             System.out.println(number + " is divisible by 2 or 3.");
17
18         if (number % 2 == 0 ^ number % 3 == 0)
19             System.out.println(number +
20                 " is divisible by 2 or 3, but not both.");
21     }
22 }
```

or

exclusive or

Program 4: TestBooleanOperators

Enter an integer:



Program 4: TestBooleanOperators

Enter an integer: 4



Program 4: TestBooleanOperators

```
Enter an integer: 4   
4 is divisible by 2 or 3.  
4 is divisible by 2 or 3, but not both.
```



Program 4: TestBooleanOperators

```
Enter an integer: 4   
4 is divisible by 2 or 3.  
4 is divisible by 2 or 3, but not both.
```



```
Enter an integer: 
```



Program 4: TestBooleanOperators

```
Enter an integer: 4   
4 is divisible by 2 or 3.  
4 is divisible by 2 or 3, but not both.
```



```
Enter an integer: 18
```



Program 4: TestBooleanOperators

Enter an integer: 4
4 is divisible by 2 or 3.
4 is divisible by 2 or 3, but not both.



Enter an integer: 18
18 is divisible by 2 and 3.
18 is divisible by 2 or 3.



Logical Operators

- Final notes:
 - If one of the operands of an `&&` operator is `false`, then the expression is `false`.
 - If one of the operands of an `||` operator is `true`, then the expression is `true`.
- Java uses these properties to improve the performance of these operators
 - Example:
 - When evaluating `p1 && p2`, Java first evaluates `P1`
 - If `p1` is `true`, Java will evaluate `p2`
 - But if `p1` is `false`, Java will not even check `p2`

Logical Operators

- Caution:
 - In mathematics, the following expression is correct:
`1 <= daysInMonth <= 31`
 - However, it is not correct in Java, because `1 <= daysInMonth` is evaluated to a `boolean` value, which cannot be compared to 31
 - The correct expression in Java is
`(1 <= daysInMonth) && (daysInMonth <= 31)`

Check Point

- Assuming that x is 1, show the result of the following Boolean expressions:
 - $(\text{true}) \ \&\& \ (3 > 4)$
 - $!(x > 0) \ \&\& \ (x > 0)$
 - $(x > 0) \ || \ (x < 0)$
 - $(x \neq 0) \ || \ (x == 0)$
 - $(x \geq 0) \ || \ (x < 0)$
 - $(x \neq 1) == !(x == 1)$

Check Point

- Assuming that x is 1, show the result of the following Boolean expressions:

- a. $(\text{true}) \ \&\& \ (3 > 4)$ false
- b. $!(x > 0) \ \&\& \ (x > 0)$ false
- c. $(x > 0) \ || \ (x < 0)$ true
- d. $(x \neq 0) \ || \ (x == 0)$ true
- e. $(x \geq 0) \ || \ (x < 0)$ true
- f. $(x \neq 1) == !(x == 1)$ true

Check Point

- Write a Boolean expression that evaluates to true if a number stored in variable num is between 1 and 100.
 - Solution:
- Write a Boolean expression that evaluates to true if a number stored in variable num is between 1 and 100 or the number is negative.
 - Solution:

Check Point

- Write a Boolean expression that evaluates to true if a number stored in variable num is between 1 and 100.

- Solution:

`(1 <= num) && (num <= 100)`

- Write a Boolean expression that evaluates to true if a number stored in variable num is between 1 and 100 or the number is negative.

- Solution:

`(1 <= num) && (num <= 100) || (num < 0)`

Program 5: Leap Year

- Write a program that determines if a given year is a leap year. Specifically, ask the user to enter a year. Then determine if that year is a leap year and display the results.
- Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 5: Leap Year

- Step 1 : Problem-solving Phase
 - Notes:
 - What is a leap year?
 - A leap year has 366 days (instead of 365)
 - Why?
 - The earth takes approximately 365.25 days to circle around the sun
 - However, the Gregorian year has only 365 days
 - Therefore, every four years, the number of days is increased to 366
 - A year is a leap year if it is divisibly by 4 but not by 100, or if it is divisible by 400.

Program 5: Leap Year

- Step 1 : Problem-solving Phase
 - Notes:
 - Which years are leap years?
 - There are three criteria:
 1. The year must be evenly divisibly by 4
 2. If the year can be evenly divided by 100, then it is NOT a leap year, unless...
 3. The year is also evenly divisible by 400. Then it is a leap year.
 - Summary: a year is a leap year if it is divisibly by 4 but not by 100, or if it is divisible by 400.

Program 5: Leap Year

- Step 1 : Problem-solving Phase
 - Notes:
 - Summary: a year is a leap year if it is divisibly by 4 but not by 100, or if it is divisible by 400.
 - We need to write a Boolean expression to determine if a given variable, year, is a leap year.
 - Consider how to convert the above condition into a Boolean expression.
 - Solution:

```
boolean isLeapYear =  
(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
```

Program 5: Leap Year

- Step 1 : Problem-solving Phase
- Algorithm:
 1. Prompt user for input
 2. Scan the year from user
 3. Determine if it is a leap year
 4. Display results to user

Program 5: Leap Year

LISTING 3.7 LeapYear.java

```
1  import java.util.Scanner;
2
3  public class LeapYear {
4      public static void main(String[] args) {
5          // Create a Scanner
6          Scanner input = new Scanner(System.in);
7          System.out.print("Enter a year: ");
8          int year = input.nextInt();           input
9
10         // Check if the year is a leap year
11         boolean isLeapYear =                 leap year?
12             (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
13
14         // Display the result
15         System.out.println(year + " is a leap year? " + isLeapYear);    display result
16     }
17 }
```

Program 5: Leap Year

Enter a year: 2008



Enter a year: 1900



Program 5: Leap Year

Enter a year: 2008
2008 is a leap year? true



Enter a year: 1900
1900 is a leap year? false



Program 6: HexDigit2Dec

- Write a program that prompts the user to enter one hex digit and then displays this as a decimal value.
- Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 6: HexDigit2Dec

- Step 1 : Problem-solving Phase
- Algorithm:
 1. Prompt the user to enter a hex digit
 2. Check to see if the input is exactly one digit
 3. If so, confirm the input is between 0-9 or A-F
 - Then print the decimal equivalent
 4. Otherwise, print invalid input
- For step 3 above, we can use methods we've learned in this Chapter

Program 6: HexDigit2Dec

- Step 2:
Implementation

Note:

- `System.exit(1)` terminates the program.
- `isDigit(ch)` methods in `Character` class that returns true if the specified character is a digit.

LISTING 4.4 HexDigit2Dec.java

```
1  import java.util.Scanner;
2
3  public class HexDigit2Dec {
4      public static void main(String[] args) {
5          Scanner input = new Scanner(System.in);
6          System.out.print("Enter a hex digit: ");
7          String hexString = input.nextLine();
8
9          // Check if the hex string has exactly one character
10         if (hexString.length() != 1) {
11             System.out.println("You must enter exactly one character");
12             System.exit(1);
13         }
14
15         // Display decimal value for the hex digit
16         char ch = hexString.charAt(0);
17         if (ch <= 'F' && ch >= 'A') {
18             int value = ch - 'A' + 10;
19             System.out.println("The decimal value for hex digit "
20                 + ch + " is " + value);
21         }
22         else if (Character.isDigit(ch)) {
23             System.out.println("The decimal value for hex digit "
24                 + ch + " is " + ch);
25         }
26         else {
27             System.out.println(ch + " is an invalid input");
28         }
29     }
30 }
```



VideoNote
Convert hex t

input string

check length

is A-F?

is 0-9?

Program 6: HexDigit2Dec

Enter a hex digit: AB7C



Enter a hex digit: B



Enter a hex digit: 8



Enter a hex digit: T



Program 6: HexDigit2Dec

Enter a hex digit: AB7C ↵ Enter
You must enter exactly one character



Enter a hex digit: B ↵ Enter
The decimal value for hex digit B is 11



Enter a hex digit: 8 ↵ Enter
The decimal value for hex digit 8 is 8



Enter a hex digit: T ↵ Enter
T is an invalid input



switch Statements

- Nested if statements are very helpful
 - They allow us to fully translate our logical conditions into code
- However, overuse of nested if statements can make a program difficult to read.
- Solution
 - Java provides a **switch** statement to simplify coding for multiple conditions

switch Statements

- Here is an example switch statement:

```
switch (status) {  
    case 0: code HERE for case 0;  
        break;  
    case 1: code HERE for case 1;  
        break;  
    case 2: code HERE for case 2;  
        break;  
    case 3: code HERE for case 3;  
        break;  
    default: System.out.println("Error: invalid status");  
        System.exit(1);  
}
```

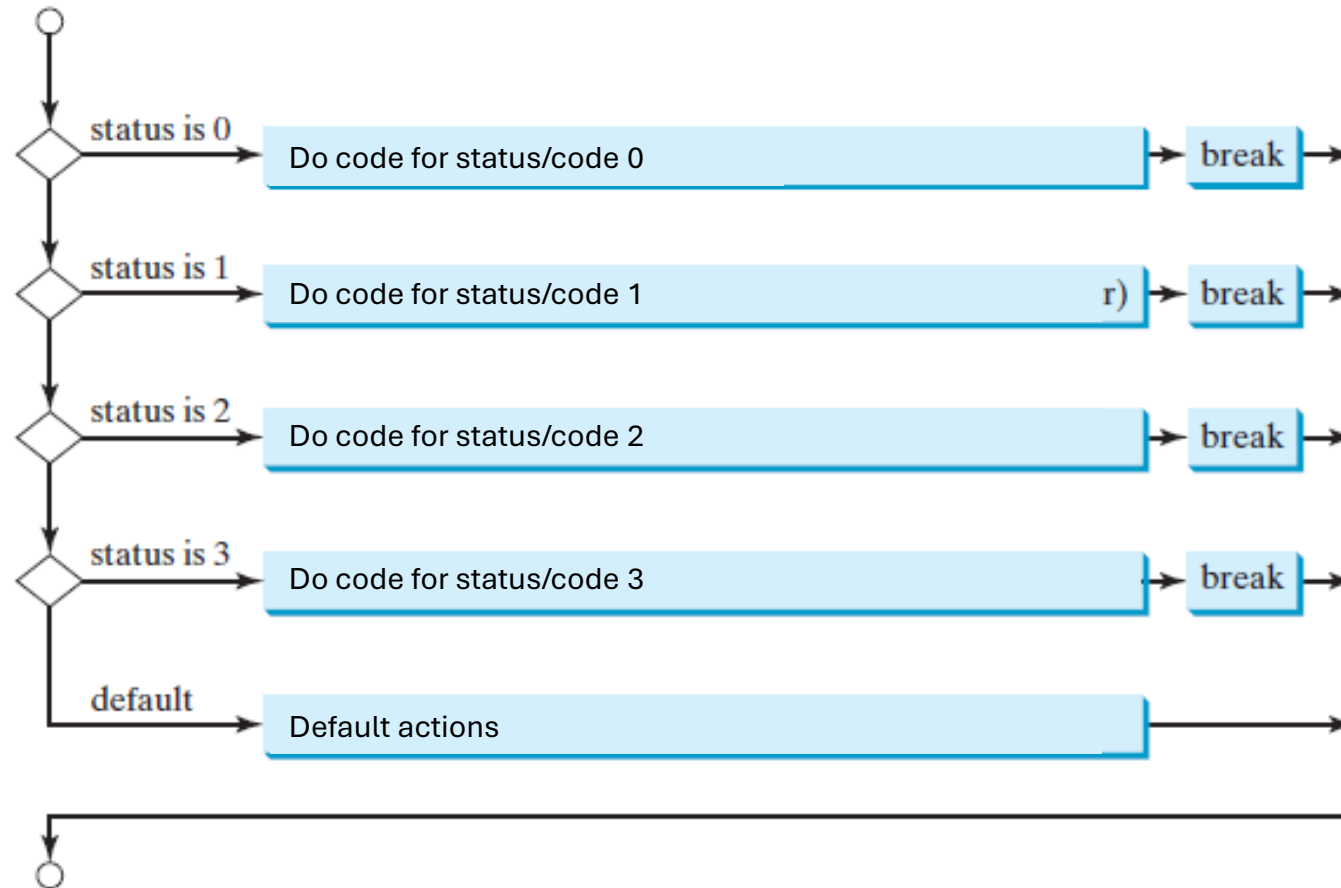
Here, status is an int representing a user choice

switch Statements

- So this `switch` statement checks to see whether `status` matches the value `0`, `1`, `2`, or `3`, and in that exact order.
- If the `status` matches a value, the code for that `status/case` is executed.
- Then, just like an `if` statement, the rest of the code is skipped.
 - This is done with the `break`.
 - This will break us out of the `switch` block.

switch Statements

- Here is the flowchart for the previous code:



switch Statements

- Syntax for the switch statement:

```
switch (switch-expression) {  
    case value1: statement(s)1;  
        break;  
    case value2: statement(s)2;  
        break;  
    ...  
    case valueN: statement(s)N;  
        break;  
default: statement(s)-for-default;  
}
```

switch Statements

- switch statement observes the following rules:
 - **switch**-expression must yield a value of **char**, **byte**, **short**, **int**, or **String** type.
 - The char and String types will be introduced in the next chapter.
 - **switch**-expression must always be enclosed in parentheses

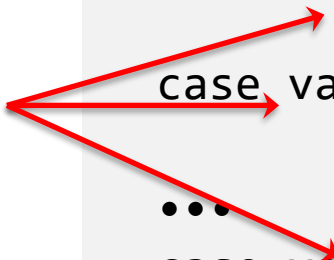


```
switch (switch-expression) {  
    case value1: statement(s)1;  
                break;  
    case value2: statement(s)2;  
                break;  
    ...  
    case valueN: statement(s)N;  
                break;  
    default: statement(s)-for-default;  
}
```

switch Statements

- switch statement observes the following rules:
 - The `value1`, ..., and `valueN` must have the same data type as the value of the `switchexpression`.
 - Note that `value1`, ..., and `valueN` are constant expressions
 - Meaning that they cannot contain variables, such as `1 + x`.

```
switch (switch-expression) {  
    case value1: statement(s)1;  
                break;  
    case value2: statement(s)2;  
                break;  
    ...  
    case valueN: statement(s)N;  
                break;  
    default: statement(s)-for-default;  
}
```



switch Statements

- switch statement observes the following rules:
 - When the value in a **case** statement matches the value of the **switch** expression, the statements starting from this case are executed.
 - Execution will continue until **either** a **break** statement or the end of the **switch** statement is reached.

```
switch (switch-expression) {  
    case value1: statement(s)1;  
                break;  
    case value2: statement(s)2;  
                break;  
    ...  
    case valueN: statement(s)N;  
                break;  
    default: statement(s)-for-default;  
}
```

switch Statements

- switch statement observes the following rules:
 - The keyword **break** is optional, but it should be used at the end of each case in order to terminate the remainder of the switch statement.
 - If the **break** statement is not present, the next case statement will be executed.
 - The **default** case, which is optional, can be used to perform actions when none of the specified cases matches the **switch-expression**

```
switch (switch-expression) {  
    case value1: statement(s)1;  
                break;  
    case value2: statement(s)2;  
                break;  
    ...  
    case valueN: statement(s)N;  
                break;  
    default: statement(s)-for-default;  
}
```

switch Statements

- With an `if/else-if` block, once a condition is met
 - the code for the condition is executed, and
 - the rest of the `if/else-if` block is skipped.
- In this same way, with a `switch` statement, once a case is correctly found, the code for that case should be executed, and then the rest of the switch block should be skipped
 - How do we do this?
 - We use a `break` statement

switch Statements

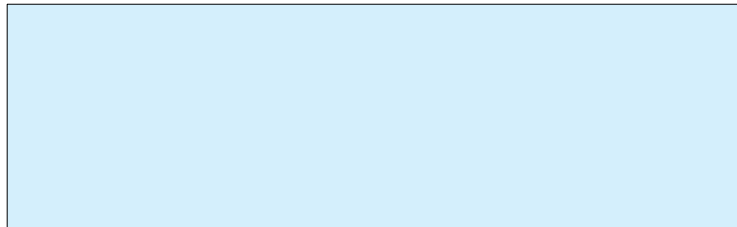
- **break** statement:
 - Usually you will want to use a break statement at the end of each case.
 - This will skip all other cases in the switch statement
 - If you do not use a break statement, the next case will be executed.
 - Sometimes, depending on your logic, you may not want to use a break at the end of each case

Trace **switch** statement

```
int a = 1;
switch (a) {
    case 1: System.out.println("ONE");
    case 7: System.out.println("SEVEN");
    case 2: System.out.println("THREE");
}
```

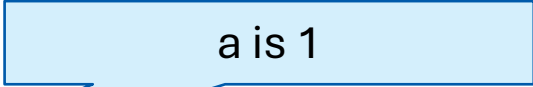
...

Output



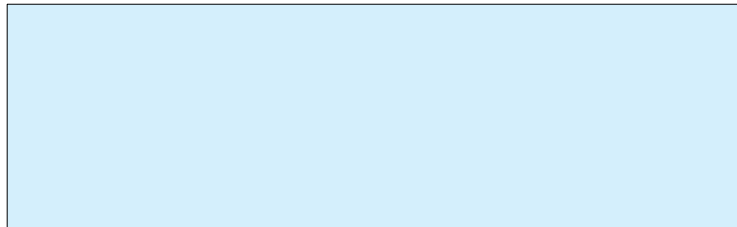
Trace **switch** statement

```
int a = 1;  
switch (a) {  
    case 1: System.out.println("ONE");  
    case 7: System.out.println("SEVEN");  
    case 2: System.out.println("THREE");  
}
```



...

Output



Trace **switch** statement

Execute this line

```
int a = 1;  
switch (a) {  
    case 1: System.out.println("ONE");  
    case 7: System.out.println("SEVEN");  
    case 2: System.out.println("THREE");  
}
```

...

Output

ONE

Trace **switch** statement

Execute this line

```
int a = 1;  
switch (a) {  
    case 1: System.out.println("ONE");  
    case 7: System.out.println("SEVEN");  
    case 2: System.out.println("THREE");  
}
```

...

Output

```
ONE  
SEVEN
```

Trace **switch** statement

Execute this line

```
int a = 1;  
switch (a) {  
    case 1: System.out.println("ONE");  
    case 7: System.out.println("SEVEN");  
    case 2: System.out.println("THREE");  
}
```

...

Output

```
ONE  
SEVEN  
THREE
```

Trace **switch** statement

Execute next statement

```
int a = 1;  
switch (a) {  
    case 1: System.out.println("ONE");  
    case 7: System.out.println("SEVEN");  
    case 2: System.out.println("THREE");  
}
```

...

Output

ONE
SEVEN
THREE

Trace **switch** statement (**break**)

Pay attention:

We now use “break” to exit the switch statement.

```
int a = 1;
switch (a) {
    case 1: System.out.println("ONE");
            break;
    case 7: System.out.println("SEVEN");
            break;
    case 2: System.out.println("THREE");
            break;
}
```

...

Output

Trace **switch** statement (**break**)

a is 1

```
int a = 1;  
switch (a) {  
    case 1: System.out.println("ONE");  
            break;  
    case 7: System.out.println("SEVEN");  
            break;  
    case 2: System.out.println("THREE");  
            break;  
}
```

...

Output

Trace **switch** statement (**break**)

Execute this line

```
int a = 1;  
switch (a) {  
    case 1: System.out.println("ONE");  
            break;  
    case 7: System.out.println("SEVEN");  
            break;  
    case 2: System.out.println("THREE");  
            break;  
}
```

...

Output

ONE

Trace **switch** statement (**break**)

Execute this line

```
int a = 1;
switch (a) {
    case 1: System.out.println("ONE");
           break;
    case 7: System.out.println("SEVEN");
           break;
    case 2: System.out.println("THREE");
           break;
}
```

...

Output

ONE

Trace **switch** statement (**break**)

```
int a = 1;
switch (a) {
    case 1: System.out.println("ONE");
            break;
    case 7: System.out.println("SEVEN");
            break;
    case 2: System.out.println("THREE");
            break;
}
```

Execute next statement

...

Output

ONE

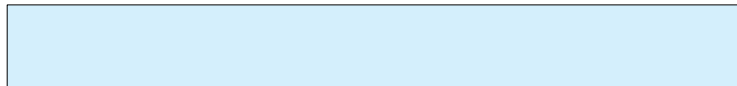
Trace **switch** statement (**break**)

Pay attention:

Here, we used the default option, which is used for any case not listed.

```
int a = 10;
switch (a) {
    case 1: System.out.println("ONE");
            break;
    case 7: System.out.println("SEVEN");
            break;
    case 2: System.out.println("THREE");
            break;
    default: System.out.println("ERROR");
}
```

Output



Trace **switch** statement (**break**)

Execute this line

```
int a = 10;
switch (a) {
    case 1: System.out.println("ONE");
            break;
    case 7: System.out.println("SEVEN");
            break;
    case 2: System.out.println("THREE");
            break;
    default: System.out.println("ERROR");
}
```

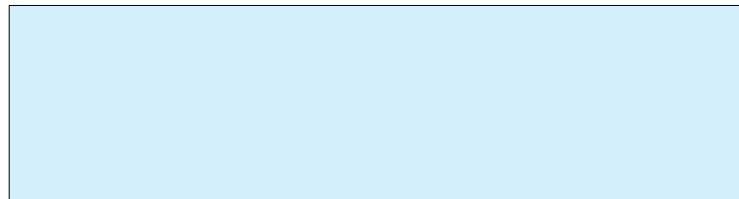
Output

ERROR

Trace **switch** statement

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
  
    case 'B': System.out.println(ch);  
  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

Output

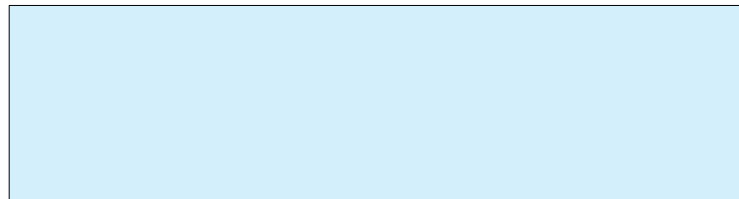


Trace **switch** statement

ch is 'a'

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
  
    case 'B': System.out.println(ch);  
  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

Output



Trace **switch** statement

Execute this line

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
    case 'B': System.out.println(ch);  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

Output

a

Trace **switch** statement

Execute this line

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
    case 'B': System.out.println(ch);  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

Output

a
a

Trace **switch** statement

Execute this line

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
    case 'B': System.out.println(ch);  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

Output

a
a
a

Trace **switch** statement

Execute next statement

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
  
    case 'b': System.out.println(ch);  
  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

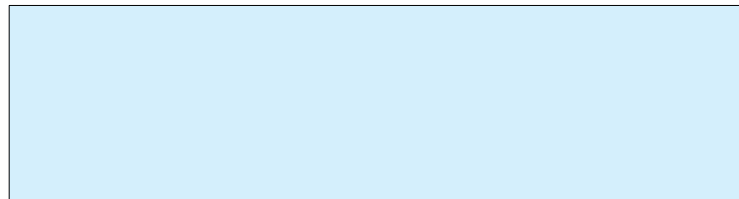
Output

a
a
a

Trace **switch** statement

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
                break;  
    case 'B': System.out.println(ch);  
                break;  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

Output

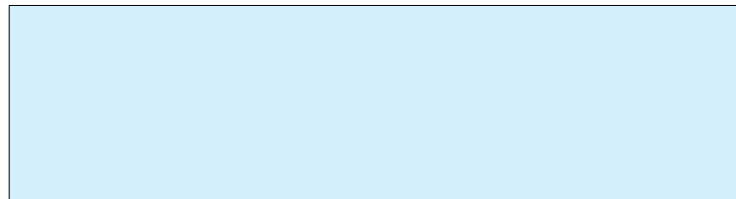


Trace **switch** statement

ch is 'a'

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
                break;  
    case 'B': System.out.println(ch);  
                break;  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

Output



Trace **switch** statement

Execute this line

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
                break;  
    case 'B': System.out.println(ch);  
                break;  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

Output

a

Trace **switch** statement

Execute this line

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
    break;  
    case 'B': System.out.println(ch);  
    break;  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

Output

a

Trace **switch** statement

Execute next statement

```
char ch = 'a';  
switch (ch) {  
    case 'a': System.out.println(ch);  
                break;  
    case 'b': System.out.println(ch);  
                break;  
    case 'c': System.out.println(ch);  
}  
... // Next statement;
```

Output

a

Trace **switch** statement

Suppose day is 2

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```

Trace **switch** statement

Match case 2

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```

Trace **switch** statement

Fall through case 3

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```

Trace **switch** statement

Fall through case 4

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```

Trace **switch** statement

Fall through case 5

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```

Trace **switch** statement

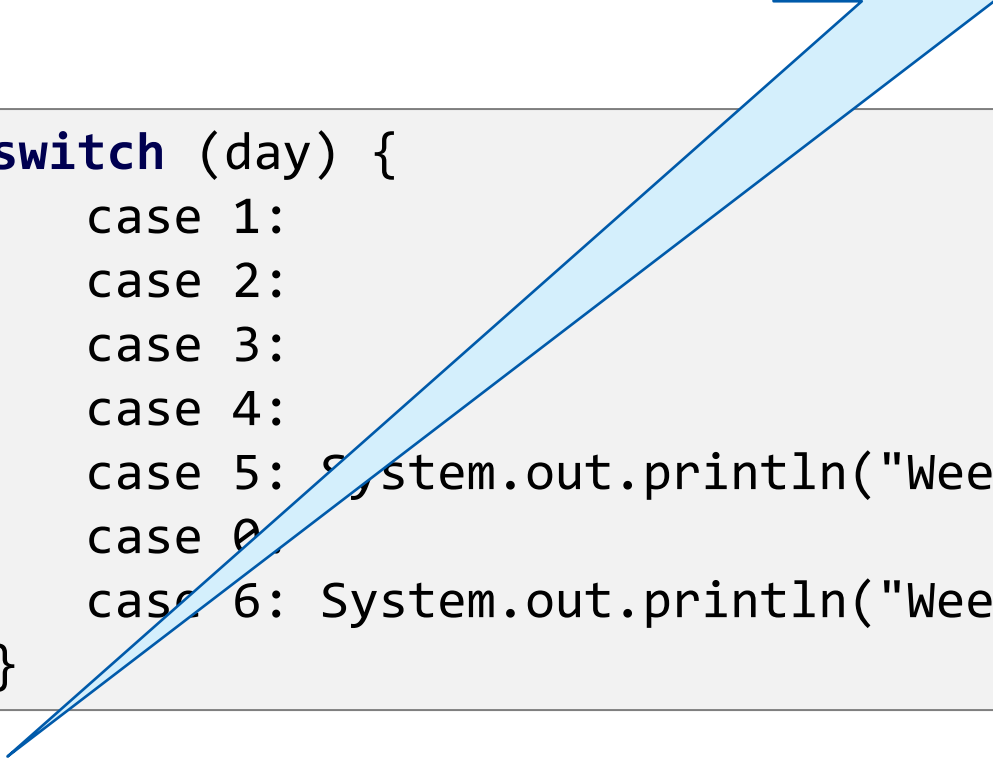
Encounter break

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```

Trace **switch** statement

Exit the statement

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 6:  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```



Check Point

- Write a `switch` statement that displays Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday if day is 0, 1, 2, 3, 4, 5, 6, respectively.
- Solution:

Check Point

- Write a `switch` statement that displays Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday if `day` is 0, 1, 2, 3, 4, 5, 6, respectively.

- Solution:

```
switch (day) {
```

```
}
```

Check Point

- Write a `switch` statement that displays Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday if day is 0, 1, 2, 3, 4, 5, 6, respectively.

- Solution:

```
switch (day) {  
    case 0:  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5:  
    case 6:  
}
```

Check Point

- Write a `switch` statement that displays Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday if day is 0, 1, 2, 3, 4, 5, 6, respectively.

- Solution:

```
switch (day) {  
    case 0: System.out.println("Sunday"); break;  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5:  
    case 6:  
}
```

Check Point

- Write a `switch` statement that displays Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday if day is 0, 1, 2, 3, 4, 5, 6, respectively.

- Solution:

```
switch (day) {  
    case 0: System.out.println("Sunday"); break;  
    case 1: System.out.println("Monday"); break;  
    case 2: System.out.println("Tuesday"); break;  
    case 3: System.out.println("Wednesday"); break;  
    case 4: System.out.println("Thursday"); break;  
    case 5: System.out.println("Friday"); break;  
    case 6: System.out.println("Saturday"); break;  
}
```

Conditional Expressions

- You may want to assign a value to a variable based on certain conditions
- Example:
 - The following statement assigns **1** to **y** if **x > 0**, and it assigns **-1** to **y** if **x <= 0**.
 - Code:

```
if (x > 0)
    y = 1;
else
    y = -1;
```

Conditional Expressions

- The last code works just fine
- But you can use a conditional expression to achieve the same result:

```
y = (x > 0) ? 1 : -1;
```

- The generic syntax is as follows:
boolean-expression ? expression1 : expression2;
- The result of this conditional expression is
 - `expression1` if `boolean-expression` is `true`;
 - Otherwise, the result is `expression2`.

Conditional Expressions

- Example:
 - Given two numbers, `num1` and `num2`, save the larger into a variable called `max`.

- You can do this with an `if` statement

```
if (num 1 > num2)
    max = num1;
else
    max = num2;
```

- Or you can use one conditional expression as follows:

Conditional Expressions

- Example:
 - Given two numbers, `num1` and `num2`, save the larger into a variable called `max`.

- You can do this with an `if` statement

```
if (num1 > num2)
    max = num1;
else
    max = num2;
```

- Or you can use a conditional expression as follows:

```
max = (num1 > num2) ? num1 : num2;
```

Conditional Expressions

- Example:

- Given a variable, num, display the message “num is even” if num is even; otherwise, display “num is odd”
- You can do this with an if/else statement

```
if (num % 2 == 0)
    System.out.println("num is even");
else
    System.out.println("num is odd");
```

- `System.out.println((num % 2 == 0) ? "num is even" : "num is odd");`

Check Point

- Rewrite the following `if` statements using the conditional operator.

```
if (ages >= 16)
    ticketPrice = 20;
else
    ticketPrice = 10;
```

- Solution

Check Point

- Rewrite the following `if` statements using the conditional operator.

```
if (ages >= 16)
    ticketPrice = 20;
else
    ticketPrice = 10;
```

- Solution

```
ticketPrice = (ages >= 16) ? 20 : 10;
```

Operator Precedence

- We have introduced operator precedence involving arithmetic operators
 - Example:
 - * and / has higher precedence than + and -
- But what about expressions with other operators
 - Example:
 $3+4*4>5* (4 + 3) - 1 \ \&\& \ (4 - 3 > 5)$
 - What is the value? What is the execution order of the above example?
 - We need to know the precedence rules for this!

Operator Precedence

TABLE 3.8 Operator Precedence Chart

<i>Precedence</i>	<i>Operator</i>
	<code>var++</code> and <code>var--</code> (Postfix)
	<code>+</code> , <code>-</code> (Unary plus and minus), <code>++var</code> and <code>--var</code> (Prefix)
	<code>(type)</code> (Casting)
	<code>!</code> (Not)
	<code>*</code> , <code>/</code> , <code>%</code> (Multiplication, division, and remainder)
	<code>+</code> , <code>-</code> (Binary addition and subtraction)
	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> (Relational)
	<code>==</code> , <code>!=</code> (Equality)
	<code>^</code> (Exclusive OR)
	<code>&&</code> (AND)
	<code> </code> (OR)
	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> (Assignment operator)

Operator Precedence

- Details:
 - The expression in the parentheses is evaluated first.
 - Parentheses can be nested
 - In which case the expression in the inner parentheses is executed first.
 - When evaluating an expression without parentheses
 - The operators are applied according to the **precedence rule** and the **associativity rule**.
 - If operators with the same precedence are next to each other
 - Their associativity determines the order of evaluation.
 - All binary operators except assignment operators are left-associative.

Operator Precedence

- Details:

- Example:

- $+$ and $-$ are of the same precedence and are left associative
 - Therefore,

$a - b + c - d$ is equivalent to

- Example:

- Assignment operators are right associative

- Therefore, $a = b, += c = 5$ is equivalent to

Operator Precedence

- Details:

- Example:

- $+$ and $-$ are of the same precedence and are **left associative**
 - Therefore,

$a - b + c - d$	is equivalent to	$((a - b) + c) - d$
-----------------	------------------	---------------------

- Example:

- Assignment operators are **right associative**

$a = b + c = 5$	is equivalent to
-----------------	------------------

Operator Precedence

- Details:

- Example:

- $+$ and $-$ are of the same precedence and are **left associative**
 - Therefore,

$a - b + c - d$	is equivalent to	$((a - b) + c) - d$
-----------------	------------------	---------------------

- Example:

- Assignment operators are **right associative**

$a = b, += c = 5$	is equivalent to	$a = (b += (c = 5))$
-------------------	------------------	----------------------

Debugging

- Dealing with programming errors
 - Remember: syntax errors and runtime errors are not difficult to find
 - However, logic errors can be very challenging
 - Logic errors are called bugs
 - **Debugging** is the process of finding and correcting these logic errors

Debugging

- Methods of debugging:
 - You can hand-trace the program
 - Meaning, you try to find the error by reading the program
 - Clearly, this is very difficult and time consuming
 - You can insert print statements throughout the program
 - The print statements allow you to see how far the execution has reached
 - You can also print, and then view, the values of variables during execution of the program.
 - Again, this is time consuming

Debugging

- Methods of debugging:
 - These two methods are okay, but they are slow.
 - They really only work for small, simple programs
 - So what about large, complex programs?
 - The best solution is to use a debugger utility
- All Java IDE programs, such as Eclipse and NetBeans, include integrated debuggers.
 - Learning how to use these debuggers is very important.

Debugging

- Debugger is a program that facilitates debugging.
- You can use a debugger to
 - Execute a single statement at a time.
 - Trace into or stepping over a method.
 - Set breakpoints.
 - Display variables.
 - Display call stack.
 - Modify variables.



بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-202 Programming I

END

